

QA Strategy and Execution Document

Project: MindBridge — Shareable Content Feature

Your name	Student
Date	07 August 2026
Jira project name	MindBridge (project key: MB) — Epic: Dashboard Sharing & Export
Story under test	MB-142 — “As a learner, I want to share my dashboard with others so they can see it”

Assumptions made about the software under test

The lab environment does not specify the MindBridge technology stack, so the following assumptions were fixed before any test design and are carried consistently through all four sections below. Any assumption that later proves wrong would be raised as a change to this document rather than silently reworked.

- MindBridge is a browser-based learning platform built as a three-tier application: a React single-page front end, a REST API served by Node/Express behind an API gateway, and a PostgreSQL database.
- The feature under test is the Generate share link button on the student dashboard, delivered by story MB-142 under the epic Dashboard Sharing & Export.
- The share operation is assumed to be POST /api/v1/dashboards/{dashboardId}/share, returning HTTP 201 with a shareToken and an absolute share URL.
- Share tokens are assumed to persist in a share_tokens table (id, dashboard_id, token, created_at, expires_at, revoked_at) with only one active token permitted per dashboard.
- Supported platforms are assumed to be current Chrome, Edge and Safari on desktop, plus mobile web from 320 px viewport width upward.
- The team is a single squad of three developers, one QA engineer and one product owner. There is no dedicated DBA, so the Dev team owns migrations and holds write access to the staging database while QA holds read-only access.
- The seeded story is deliberately thin: it states no link expiry, no viewer permissions, no revocation path and no behaviour for a repeated request. These gaps were closed by the ambiguity review described in Section 2 and became the acceptance criteria in Section 3.

Section 1: Architecture and RACI Matrix

The share feature touches all three tiers of the MindBridge stack, so the matrix below is scoped to ownership of quality for each layer on story MB-142 — not to who writes the code. Every cell carries a recognised designation and no cell is left blank.

Layers in scope

- **UI** — the ShareDashboardCard component on the student dashboard: the Generate share link button, the result row holding the share URL and Copy link button, and the error toast.

- **API** — POST /api/v1/dashboards/{dashboardId}/share, served by the share-link service. Owns token creation, expiry, authorisation and the response contract.
- **DB** — the share_tokens table and its uniqueness constraint on the active token per dashboard, plus the read model that resolves a token back to a dashboard.

RACI matrix

Layer	QA Team Role	Dev Team Role
UI	A — Accountable	R — Responsible
API	R — Responsible	A — Accountable
DB	C — Consulted	R — Responsible

Key: R = Responsible, A = Accountable, C = Consulted, I = Informed.

Why each role was assigned

- **UI — QA Accountable, Dev Responsible.** The Dev team builds the ShareDashboardCard component and its unit and component tests. QA carries the acceptance decision for the dashboard UI: BDD scenario verification, responsive checks at 320, 375 and 768 px, and the cross-browser pass. If a UI defect reaches production, QA answers for the gap in coverage.
- **API — QA Responsible, Dev Accountable.** The response contract is a design artefact the Dev team owns and must answer for — status codes, payload schema and error handling. QA is responsible for executing the contract, negative and boundary tests against the endpoint and for reporting every deviation, which is exactly how MB-DEF-001 in Section 4 was found.
- **DB — QA Consulted, Dev Responsible.** QA holds read-only access to the staging database, so the Dev team runs the migration and executes the persistence checks. QA is consulted: I supply the data-integrity assertions (exactly one active token per dashboard, token uniqueness, expires_at populated) and review the evidence before sign-off.

Section 2: STLC Strategy and Environments

Testing starts at the requirements phase and ends with a release recommendation. The strategy below names the static techniques applied before code exists, the dynamic techniques applied against a running build, and the environment in which execution takes place.

Static testing technique(s)

- **Requirements review / ambiguity analysis (walkthrough).** Story MB-142 was reviewed line by line with the product owner and dev lead before a single line of code was written. The story said only that the app “should give them a link”. The review surfaced four unstated requirements — link expiry, who may view the shared dashboard, how a link is revoked, and what happens when the button is pressed twice — and each was converted into testable acceptance criteria.
- **Peer code review with static analysis.** Every pull request on the share-link branch is reviewed by a second developer, with QA reviewing the test code. A SonarQube / ESLint quality gate runs on the branch and must

report no new blocker or critical issues. This is the check aimed at unhandled promise rejections in the share service — the class of fault behind the 500 recorded in Section 4.

Dynamic testing technique(s)

- **Functional black-box testing of the BDD scenarios.** The three Given-When-Then scenarios in Section 3 are executed manually against a deployed build, using equivalence partitioning and boundary value analysis on the inputs: a dashboard with zero, one and the maximum number of widgets, and token expiry at day 0, day 7 and day 8.
- **API testing (Postman collection).** Status-code and JSON-schema validation for POST `/api/v1/dashboards/{dashboardId}/share`, including the negative partitions: unauthenticated request (expect 401), non-existent dashboard id (expect 404), dashboard belonging to another learner (expect 403), and a repeat request while an active link exists.
- **Exploratory and responsive testing.** Time-boxed exploratory sessions in Chrome DevTools device mode at 320, 375 and 768 px, charter-driven around the share flow. This is the technique that exposed the layout defect MB-DEF-002.
- **Regression and smoke testing.** The existing dashboard regression suite runs after the share feature is merged, to confirm the new component has not disturbed the progress and score widgets already on the page.

Test environment

- **Staging — the environment used for this execution.** MindBridge Staging at `https://staging.mindbridge.example.com`. It runs a production-like build against its own PostgreSQL instance seeded with anonymised learner data, with the `dashboard_share_link` feature flag enabled and outbound mail stubbed. All three BDD scenarios and both defects in Section 4 were executed and reproduced here.
- **Development (supporting).** Local and CI environments where unit, component and static-analysis checks run on each commit. Not used for acceptance evidence.
- **Production (supporting).** Post-release smoke checks only, using a dedicated monitoring account. No test data is created here.

Section 3: BDD Acceptance Criteria

Story MB-142 was updated in Jira with three Given-When-Then scenarios covering a happy path, an edge case and an error condition. The screenshot below shows all three in the ticket's description field.

Your work
Projects
Filters
Dashboards
Teams

Search

Create

D

Projects / MindBridge / MB-142

As a learner, I want to share my dashboard with others so they can see it

Attach
Add a child issue
Link
Create branch
...

Description

We are adding a Generate share link button to the student dashboard. When the learner uses it, the app should give them a link they can send to someone else.

Acceptance Criteria (BDD)

Scenario 1 — Happy path: Learner generates a share link

Given I am signed in as a learner and my MindBridge dashboard has loaded

When I select the “Generate share link” button on the dashboard

Then a unique read-only share URL is displayed within 3 seconds

And the API responds with HTTP 201 and a shareToken value

Scenario 2 — Edge case: Second link requested for the same dashboard

Given I am a learner who already has an active, unexpired share link

When I select “Generate share link” a second time

Then the existing share URL is returned unchanged

And no duplicate share token is created in the database

Scenario 3 — Error condition: Share service is unavailable

Given the share-link service is unavailable

When I select “Generate share link”

Then I see the message “We couldn’t create your link. Please try again.”

And no share URL is displayed and no token is stored

In Progress

Details

Assignee	Dennis (QA)
Reporter	Product Owner
Labels	qa-lab, bdd, sharing
Parent	Dashboard Sharing & Export
Sprint	MindBridge Sprint 14
Priority	High
Environment	Staging

Created 3 days ago · Updated just now

Figure 1 — Jira story MB-142 with all three Given-When-Then scenarios in the description field.

Scenario transcript (for readability)

Scenario 1 — Happy path: Learner generates a share link	
Given	I am signed in as a learner and my MindBridge dashboard has loaded
When	I select the “Generate share link” button on the dashboard
Then	a unique read-only share URL is displayed within 3 seconds
And	the API responds with HTTP 201 and a shareToken value
Scenario 2 — Edge case: Second link requested for the same dashboard	
Given	I am a learner who already has an active, unexpired share link
When	I select “Generate share link” a second time
Then	the existing share URL is returned unchanged
And	no duplicate share token is created in the database
Scenario 3 — Error condition: Share service is unavailable	
Given	the share-link service is unavailable
When	I select “Generate share link”
Then	I see the message “We couldn’t create your link. Please try again.”

And

no share URL is displayed and no token is stored

Section 4: Defect Log and Execution Evidence

Execution summary

- **Scenario 1 (happy path) — Pass.** A unique read-only URL was returned and POST /share responded with HTTP 201.
- **Scenario 2 (edge case) — Fail.** The second request returned HTTP 500 instead of the existing link. Raised as MB-DEF-001.
- **Scenario 3 (error condition) — Partial.** A message is shown, but it is the generic “Something went wrong” toast rather than the agreed copy, and the underlying 500 is unhandled. Covered by MB-DEF-001.
- **Responsive pass at 320 / 375 / 768 px — Fail at 375 px.** Overlapping elements in the share result row. Raised as MB-DEF-002.

Defect 1 — title and description

MB-DEF-001 — POST /api/v1/dashboards/{id}/share returns HTTP 500 when a second share link is requested for the same dashboard

Requesting a share link for a dashboard that already holds an active, unexpired token causes an unhandled unique-constraint violation inside the share-link service. The API returns 500 Internal Server Error instead of returning the existing link with HTTP 200 or a handled conflict with HTTP 409. Because the failure is unhandled, the front end can only display the generic “Something went wrong” toast, so the learner is given no way to recover and no indication that a valid link already exists. This breaks Scenario 2 outright and leaves Scenario 3 only partially satisfied. Evidence is in Figure 2: the DevTools Network tab shows the POST to /api/v1/dashboards/8842/share returning status 500, with the error body visible in the Headers pane.

Defect 2 — title and description

MB-DEF-002 — Share-link result row overlaps the Study streak card and the fixed bottom navigation at a 375 px viewport

Once a link has been generated, the .share-result container is absolutely positioned with a fixed width of 411 px and bottom: -28px. At a 375 px viewport this produces three visible collisions: the Copy link button is clipped by the right edge of the viewport and cannot be fully tapped, the result row is drawn over the Study streak card so its value is unreadable, and the “Send a copy to my instructor” field overlaps the fixed bottom navigation so the field and the Home and Courses tap targets occupy the same band. Evidence is in Figure 3.

Defect log

Defect title	Description	Steps to reproduce	Expected result	Actual result	Severity
MB-DEF-001 — POST /share returns HTTP 500 on a repeat share request	Unhandled unique-constraint violation in the share-link service when an active token already exists. No graceful handling, so only a generic error toast is shown.	1. Sign in to staging as learner 8842 and open the dashboard. 2. Select Generate share link and wait for the URL to appear. 3. Reload the page, open DevTools > Network, select Generate share link again.	The existing unexpired share URL is returned with HTTP 200 and no second token row is created.	HTTP 500 Internal Server Error; no link displayed; generic “Something went wrong” toast.	High (S2)
MB-DEF-002 — Share result row overlaps card and bottom nav at 375 px	.share-result is absolutely positioned at width 411px, bottom:-28px, so it overflows and overlaps neighbouring elements on mobile widths.	1. Open the staging dashboard in Chrome DevTools device mode, iPhone SE (375 × 667). 2. Select Generate share link. 3. Observe the result row, the Study streak card and the bottom navigation.	The result row is constrained to the viewport width, stacks below the Study streak card, and the bottom navigation stays clear and tappable.	Copy link clipped off-screen; row covers the Study streak value; instructor field overlaps the bottom nav.	Medium (S3)

Execution evidence

Evidence 1 — API defect (MB-DEF-001). The DevTools Network tab shows the failing request highlighted: POST /api/v1/dashboards/8842/share with status 500, and Status Code: 500 Internal Server Error in the Headers pane.

Figure 2 — HTTP 500 on POST /api/v1/dashboards/8842/share, DevTools Network tab, staging environment.

Evidence 2 — UI defect (MB-DEF-002). Chrome DevTools device mode at iPhone SE (375 × 667). The annotated regions show the clipped Copy link button, the result row drawn over the Study streak card, and the instructor field overlapping the fixed bottom navigation.

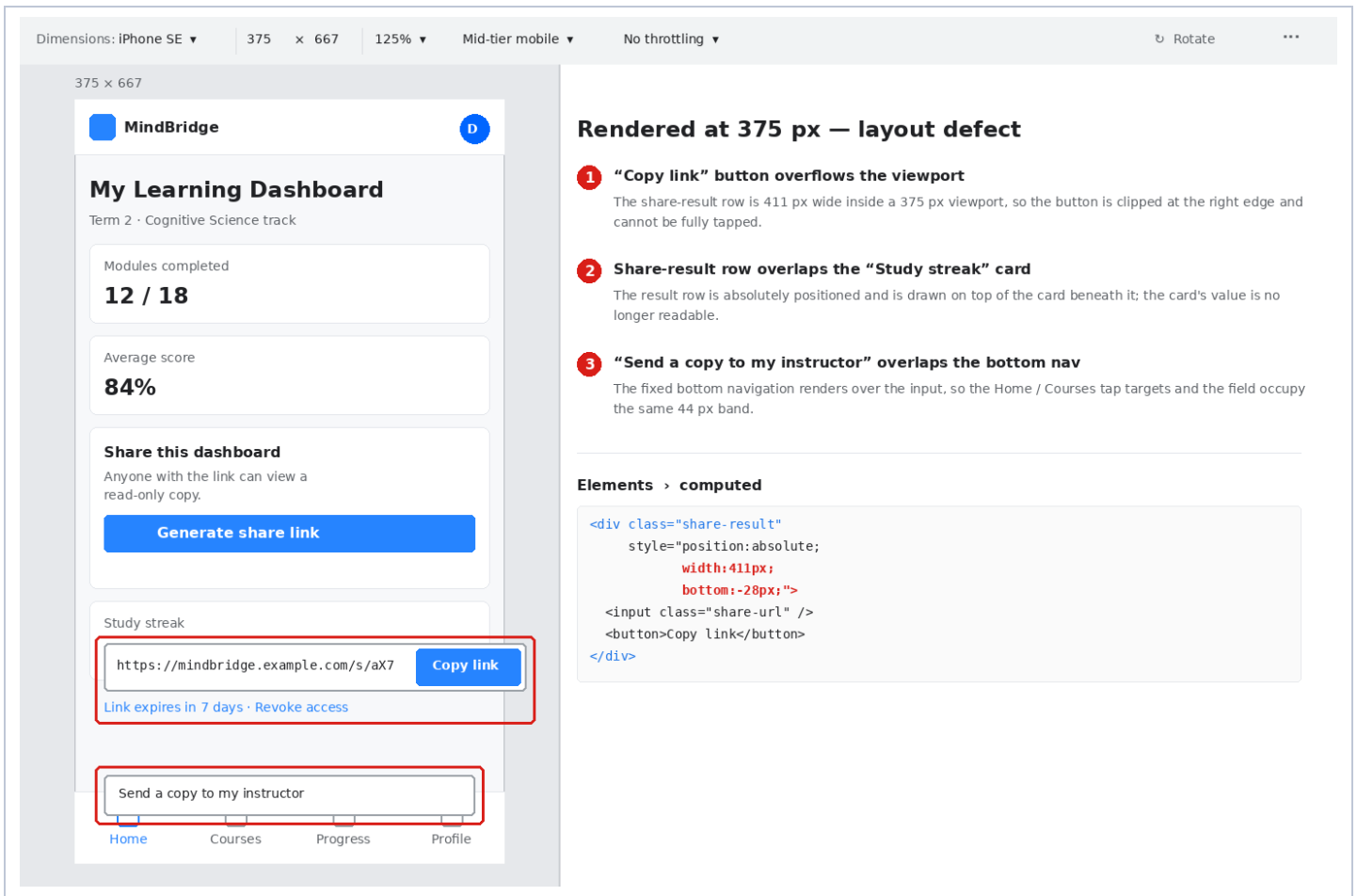


Figure 3 — Overlapping elements in a 375 px mobile viewport, staging environment.